Learning Objective

Learners will be able to...

- Describe what a language model is and how it is created
- Choose a pre-trained language model for a given application
- Use a pre-trained language model

info

Make Sure to Know

Intermediate Python.

Limitations

In this assignment, we only work with the NLTK library.

Language Models

One way to analyze natural language is to use a **language model**.

What is a Language Model?

A language model is a model which understands language – more precisely how words occur together in natural language. A language model is used to predict what word comes next.

There are a few different types of language models, including **probabilistic language models** and **machine learning language models**. Within each type of language models, there are a number of design decisions in the creation of the model, including the mechanics of the model creation (e.g. unigram vs bigram for probabilistic, Neural Network setup for machine learning).

Another design decision for a language model aside from model type is the text it is built from or trained on. Language data can come from a wide range of sources:
* chat platforms
* text repositories
* websites
* news articles
* books

Ideally, you would create or train your language model on text from the same context it will be deployed in. For example, a model trained on social media sites would be more informal and use different words than a model trained on research articles. For more general purposes, there are general purpose language models.

Choosing a Pre-Trained Language Model

The building and training of models are both complex and resource intensive. Luckily, there are **pre-trained language models**.

A couple of factors to consider when choosing a pre-trained language model:
1. What task are you using it for?
1. What are the technical requirements to use the model?

What task are you using it for?

The best place to start is to find a purpose-specific model. Pre-trained models often have descriptions which include what the pre-trained models are best for. If you cannot find a model that is specific to your task or your task is ill-defined you can use a more general purpose model.

What are the technical requirements to use the model?

While some technical requirements are easier to meet such as libraries such as PyTorch or TensorFlow, using even a pre-trained model can be resource intensive. In some cases, a minimum RAM is specified or even the use of a GPU, however even meeting the minimum hardware requirements could result in very slow results.

Overview of Popular Pre-Trained Models

There are hundreds of pre-trained language models that can be used. For an example of the breadth of models available, take a look at the list of models supported by the Hugging Face Transformers framework. A couple of well-known ones are described below but there are many more.

BERT

BERT was developed by Google, utilizing the Transformer, a novel neural network architecture. It is well suited for any task that transforms input, such as speech recognition or text-to-speech transformation.

The BERT algorithm was trained on 2,500 million Wikipedia words and 800 million words of the BookCorpus dataset. BERT is used as part of Google Search, Google Docs, and Gmail Smart Compose.

OpenAI's GPT-3

GPT-3 is a transformer-based NLP model that performs a range of tasks such as translation, question-answering, and tasks that require reasoning such as unscrambling words.

It is trained on over 175 billion parameters on 45 TB of text from all over the internet, making it one of the biggest pre-trained NLP models available. What differentiates GPT-3 from other language models is it does not require fine-tuning to perform downstream tasks, developers are allowed to reprogram the model using instructions.

Example Using DialoGPT

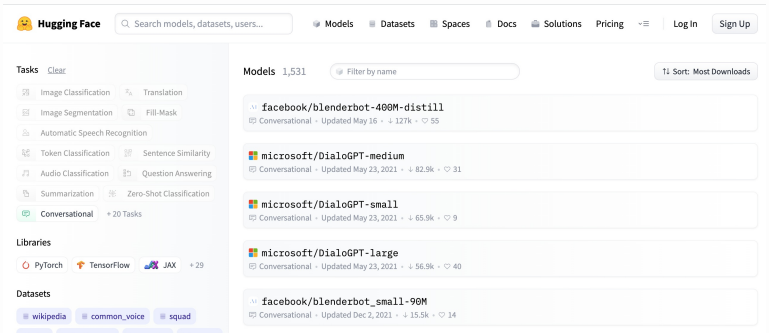
The first step in using a pre-trained model is to initialize it. We use Hugging Face's `transformers.pipeline`:

```
model = transformers.pipeline("conversational",
        model="microsoft/DialoGPT-small")
```

The first parameter is the `task` which can be any of the following:
- `"feature-extraction"`
- `"sentiment-analysis"`
- `"ner"`
- `"question-answering"`
- `"fill-mask"`
- `"summarization"`
- `"translation_xx_to_yy"`
- `"text2text-generation"`
- `"text-generation"`
- `"zero-shot-classification"`
- `"conversation"`

We are going to create a very simple chatbot example, so the task we have chosen is `"conversational"`.

The second parameter we use is the `model` parameter. As described, each model is good at different tasks, so we need to choose a model that is good at having conversations. You can check out HuggingFace's handy model list you can filter by task.



List of HugginFace models that are compatible with conversational tasks

The two most popular conversational models are Facebook's Blenderbot and Microsoft's DialoGPT – and you will notice in the code above we have specified `"microsoft/DialoGPT-small"`.

The second step is the use of the initialized model:

```
## conversation runner code
user = input("Enter text or type 'bye' to end:
        ").strip().lower()
while user != 'bye':


    # generate and print response using model
    response = str(model(transformers.Conversation(user),
        pad_token_id=50256))
    print(response[response.find("bot >> ")+6:].strip())


    # get next user input
    user = input("Enter text or type 'bye' to end:
        ").strip().lower()


# end of chat
print("Good bye!")
```

A lot of the above code is just runner code to handle the back and forth of taking input and printing output – so let's focus on the line that uses the model:

```
response = str(model(transformers.Conversation(user),
pad_token_id=50256))
```

The inner-most piece passes `user`, the user input, to the `transforomers.Conversation()` constructor which creates a `Conversation` object.

That `Conversation` object is then passed to the `model` we initialized with a unique ID number.

The model then returns a response with some other information that is removed in the following line of code.

Example Using Blenderbot

The code on the left is the same as for the DialoGPT except for the `model` specified during initialization:

```
model = transformers.pipeline("conversational",
model="facebook/blenderbot_small-90M")
```

You might notice that **the Blenderbot model is significantly slower** than the DialoGPT bot. Once you start getting responses, you might also notice that the responses seem more human-like or natural.

This is a common trade-off – the better the model, the more computational resources (space and time) it typically requires both to train and even to run.

Formative Assessment 1

Formative Assessment 2